Module 5 : Computer Memory → Data Storage

ch- Cache, Int. mem, Ext. memory

Phys type

Semiconductor → RAM, ROM, SSD

Magnetic surface → HDDs

Optical → CDs

Phys. Chars.

Volatile → power ille, data ille.

non-volatile → works w/o power.

Magnetic → non-volatile. (MM, cache)

SM
- RAM (volatile)
  - SRAM static cache
  - DRAM Dynamic. MM
  - cells
- ROM (non-volatile)

(μp CU, SP, lib)

Dram cells VS SRAM cells

Q: Why No cache memory in Von Neumann Arch?

A → At the time, tech was still developing.

Scanned by CamScanner

ROM | PROM | EPROM | EEPROM

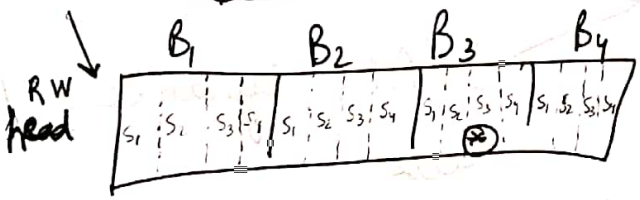
Programmable ROM

Electrically erasable PROM.

* Read definition & arch from the book *

Access methods :

• Sequential	• Direct	• Random	• Associative
start at beginning & read in order casettes, magnetic tapes.	RW head goes to beg. of B_3 then seq. search for sub-block S_3 HDD		



Performance Parameters

- Access time (latency)
- Memory cycle time → refresh ke liye time.
- Transfer rate.

RAM : 1/cycle time

non RAM : $T_n = T_A + \frac{n}{R}$

T_A → Access time

n → no. of bits

R → transfer rate

Scanned by CamScanner

MEMORY HEIRARCHY

for same capacity → MM > Ext
expensive cheaper.

Heirarchy list

- Registers
- L_1 Cache
- L_2 Cache
- MM
- Hard drive/Mag disks
- Optical
- Mag. tape

"on-chip" cache → same chip has Mprocessor & cache.

↓ down the list

- Decreasing cost per bit
- Increasing capacity.
- Decreasing access time
- Decreasing freq. of access reqd. by processor.

CACHE

Locality of Reference → exploiting the fact that processors execute instructions ~~together~~. in sequence.

- Spatial Locality
- Temporal locality.

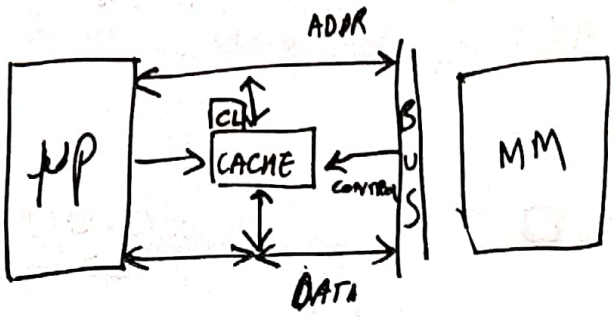
Scanned by CamScanner

6th . Nov. 19

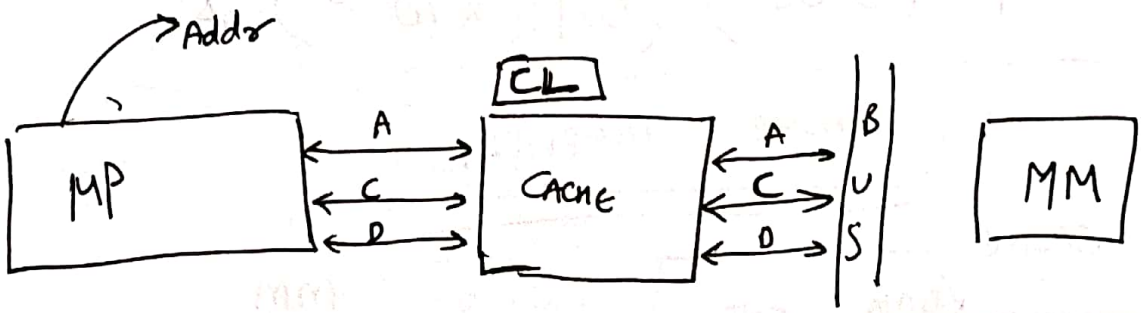
CACHE

MEMORY BOOK SE MAT KARNA

2.



1.



1. CACHE HIT → word present MISS → word not present.

HIT RATIO : $\frac{\text{CACHE HITS}}{\text{CACHE HITS + CACHE MISSES}}$

In case of cache miss, address is placed on address bus, data bus pe word and cache is updated.

Time: from MM to cache + cache to μP.

2. μP & MM can directly interact

↓

this extra time is reduced.

if the CPU requests word from MM

$$\boxed{\text{Avg access time}} = H * T_C + (1-H)(T_C + T_M)$$

H → Hit Ratio

T_C → Time taken to access word from cache

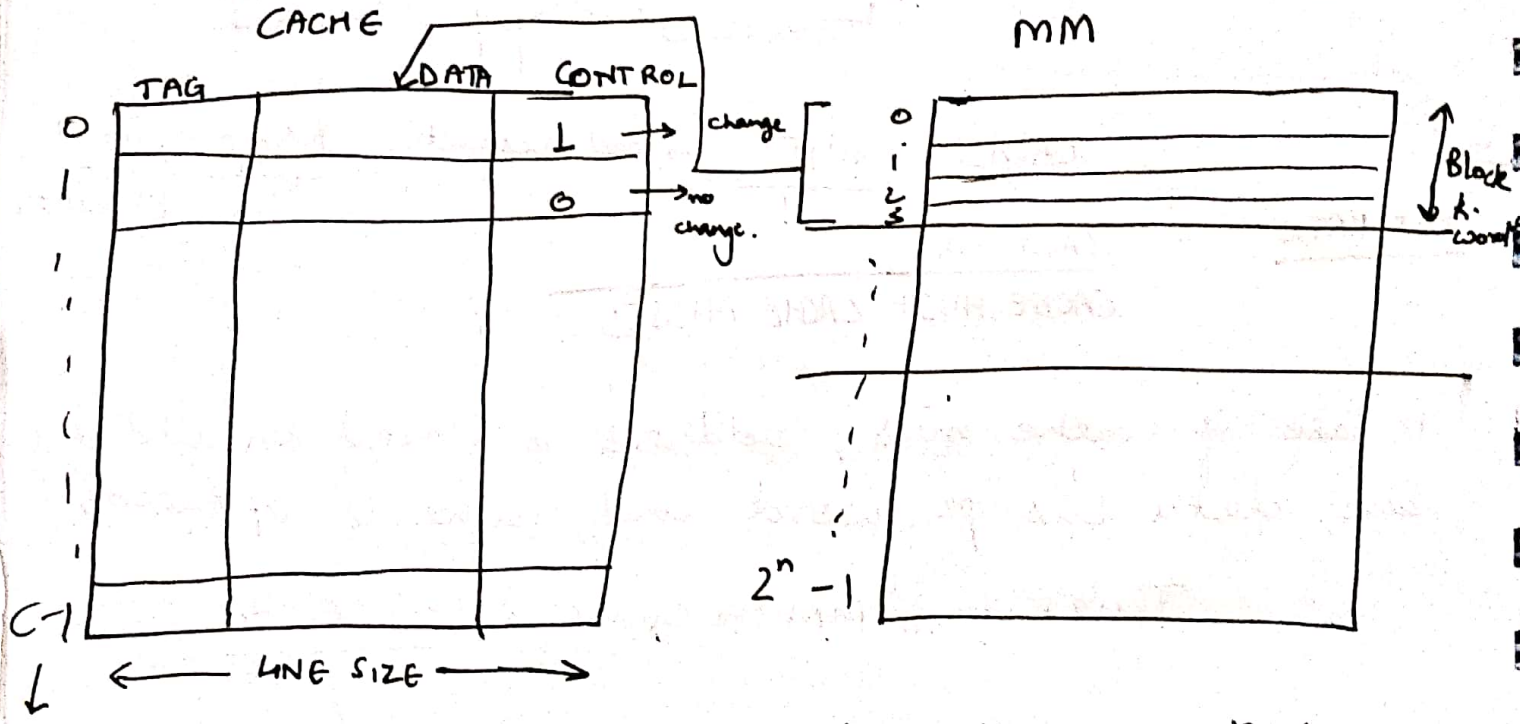
T_M → Time taken to transfer word from MM to cache

Scanned by CamScanner

Q:// Proc. has access to 2 levels of memory. L_1 has 1000 words & access time 0.01 μs. L_2 100,000 words & has access time of 0.1 μs. 95% of the cases, word is in level 1. Avg time?

$$t = 0.95 * 0.01 \times 10^{-6} + (0.05)(0.11 \times 10^{-6}) s = 0.015\ \mu s$$

CACHE MAPPING



Cache lines

# of blocks = $2^n / k$

1 block mapped to 1 cache line.

3 Mapping techniques → Direct
→ Associative
→ Set Associative.

Scanned by CamScanner

Direct: 1 block to 1 line. But after m-1 blocks, cache does not have space.

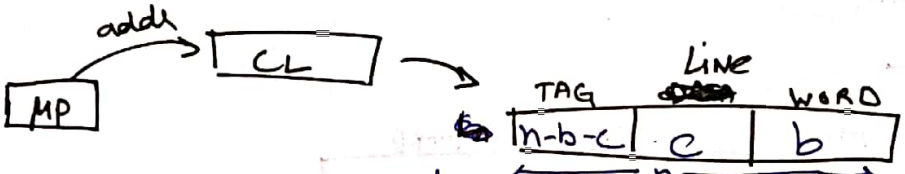
$$\boxed{\text{Cache line no.} = (\text{Block no.}) \bmod (\text{No. of C lines})}$$

Cache controller removes B_0 & adds B_M to L_0.

B_0 B_m **Thrashing**: removal of existing line to accomodate new block to the lines

*reduces hit rate considerably * easy to implement*

Processor sends out address (n-bit) to access word.



1. CL retrieves line no. | 2. matches tag field if it is the same block or not → Hit / miss

Suppose n-bits represent the address. No. of words in 1 block is given by 2^b. No. of lines in cache → 2^c. c → bits can uniquely identify line #.

LINE → used to identify line in the cache.
WORD → identifies unique word in the block.
TAG → identifies block of MM mapped into cache.

Q:// A cache holds 64 kB. data is transferred between in blocks of 4 bytes. MM → 16 MB. each bit directly addressable by a 24-bit address. Give the address format for direct mapping, associative mapping, & 2 way set ass. mapping.

$n = 24$.

~~256 blocks~~ ~~64~~ blocks of ~~256~~ ~~words~~. ~~lines~~
4000

Block size = 4B = 2^2 ← b = 2

cache size = 64 kb = 2^{16}

MM size = 16MB = 2^{24}.

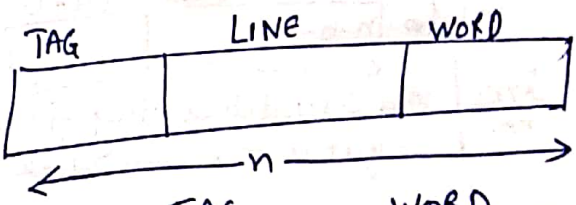
∴ # of blocks : = $2^{24}/2^2 = 2^{22}$

∴ tag = 24-14-2 = 8.

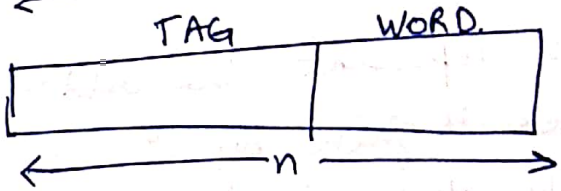
$$C = \log_2(2^{16}/2^2) = \log_2(2^{14}) = \underline{\underline{14}}$$

Cache mapping

Direct (low hit ratio)

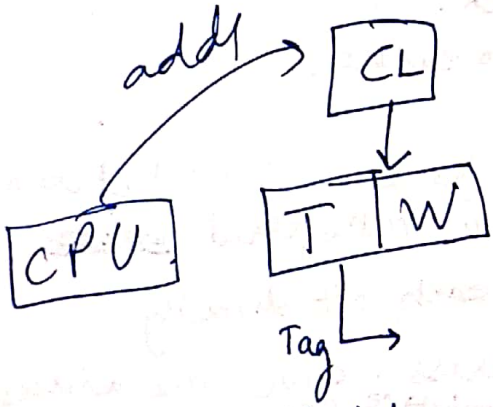


Associative



↓

any block to any line.



Tag checked with all cache lines.

This is known as associative access.

cache miss → import from M/M.

We don't know which line to throw away.

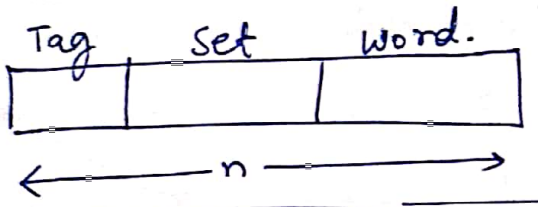
Scanned by CamScanner

Job of Cache Logic to decide which cache line needs to be replaced.

- Random → Any random
- FIFO → FIRST IN FIRST OUT: First line occupied is replaced (timer)
- LFU → Least Frequently used (note freq.)
- LRU → Least Recently used

Recently used lines move to top of cache memory. Bottom wali is LRU. Hard to implement but most relevant because of Temporal Locality of Reference

SET ASSOCIATIVE



DIRECT → 1 Line – 1 Block

ASSOCIATIVE → higher hit ratio but hard to implement

S.A. tries to incorporate positives of both.

↓

Cache divided in sets. → each sets contain few lines. no. of lines/set → same.

S.A any block to 1 set but any line of the set.

Replacement logic is also used.

Tag matching : is associative.

Scanned by CamScanner

Mapping logic → no. of sets / no. of blocks.

Q: // Block size : $4B = 2^2$

Cache size : $64{,}000\ B = 2^{16}$

MM size : $16{,}000{,}000\ B = 2^{24}$

~~ii) But cache is 2-way ∴ 2^{15}~~

~~∴ no. of sets = 2^{15}~~

~~S-A. $C = \log_2(2^{15}/2^2) = 13$~~

~~tag = 24 − 13 − 2 = 9~~

i) Associative.

TAG	Word
24 − 2 = 22	2

16 bits for cache lines.

ii) cache is 2-way

∴ 15 bits for set.

$\text{Word} = \log_2(2^{15}/2^2) = 13$

TAG	set	word
9	13	2

Scanned by CamScanner

Q:// MM → 2^{16} B Block size → 2^3 B

a) Direct mapped cache of 32 lines. (2^5 lines) What is the memory format

Into what line would the following addresses be stored

i) 0001 0001 0001 1011

ii) 1100 0011 0011 0100.

b) Consider the same system w/ 4-way set Associative memory and give the address format.

Into what address would the following addresses be stored?

i) 1101 0000 0001 1101

2) 1010 1010 1010 1010

c) How many total bytes of memory can be stored in this cache.

A:// Block size 2^3

	Tag	Line	Word
Direct	16 − 8 = 8.	5	3

	T	S	W
4-way set Associative	16−6=10	3	3

Scanned by CamScanner